

dbxmetagen

Discovering the semantic layer hidden in your data — automatically

The problem: meaning, lost at scale

2,000

tables

× 90

columns each

180,000

undocumented columns

Documenting just ONE of these tasks by hand ≈ 5 FTE-years

The meaning is already latent in your data – in the names, the values, how tables relate.
The bottleneck is purely manual effort. That is exactly what AI removes.

Why undocumented data hurts

Can't find or trust it

Analysts guess what cryptic columns like `pt_dob` actually mean — at 180,000-column scale.

Sensitive data is hidden

Personal, health, and payment data sits unlabeled — a compliance risk you can't see.

No organization

Which tables are “finance”? Which are “clinical”? Nobody tagged them, so the lake is unnavigable.

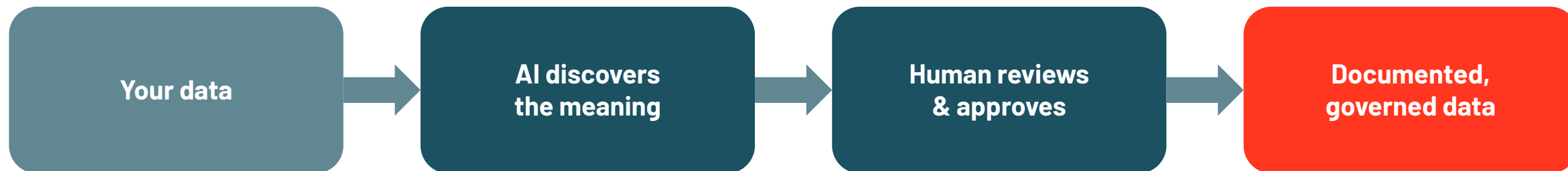
AI & BI can't help

Dashboards and AI assistants need business descriptions to work. Garbage in, garbage out.

The meaning is there — the only thing missing is the manual effort to extract it.

What dbxmetagen does

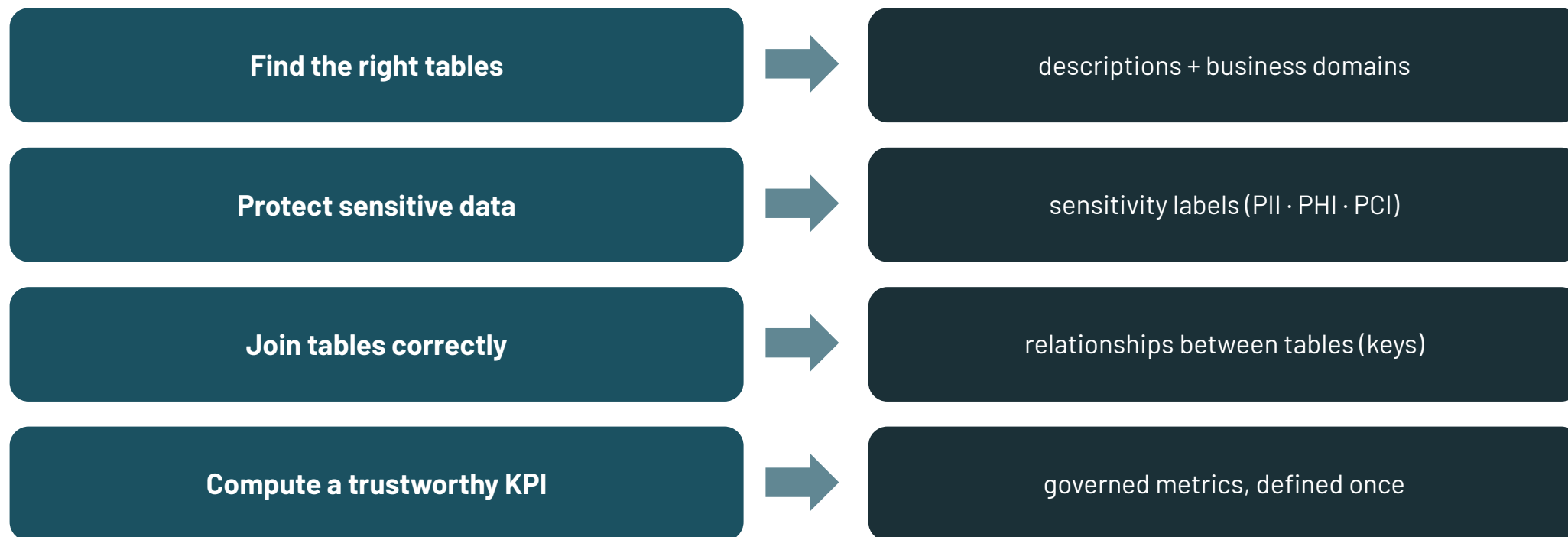
It reads your data and writes the documentation for you – and you approve it



- Reads more than names: sample data, lineage, and your own business glossary / acronyms.
- Proposes with AI (foundation models), checks itself with rules – it doesn't blindly trust the model.
- Never overwrites silently – a human reviews every change before it's applied to the catalog.
- Open-source: use it as a library in a notebook, or as an app running inside your workspace.

Different jobs need different depth

Why dbxmetagen goes beyond descriptions



Descriptions + tags get you discovery & safety. A trustworthy “quarterly sales” also needs relationships (to join) and metrics (to compute) – so dbxmetagen builds all four layers.

The building blocks

Metadata

Data about data — the catalog card for every table & column.

Unity Catalog

Where data is organized (catalog → schema → table) and governed. dbxmetagen reads from it and writes back to it.

Semantic layer

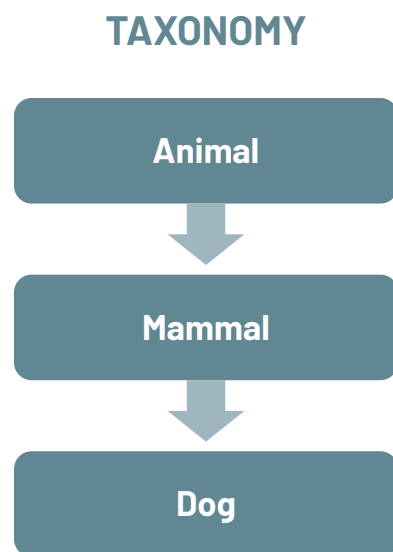
Business language over raw tables — concepts (ontology) and KPIs (metrics), defined once.

Ontology

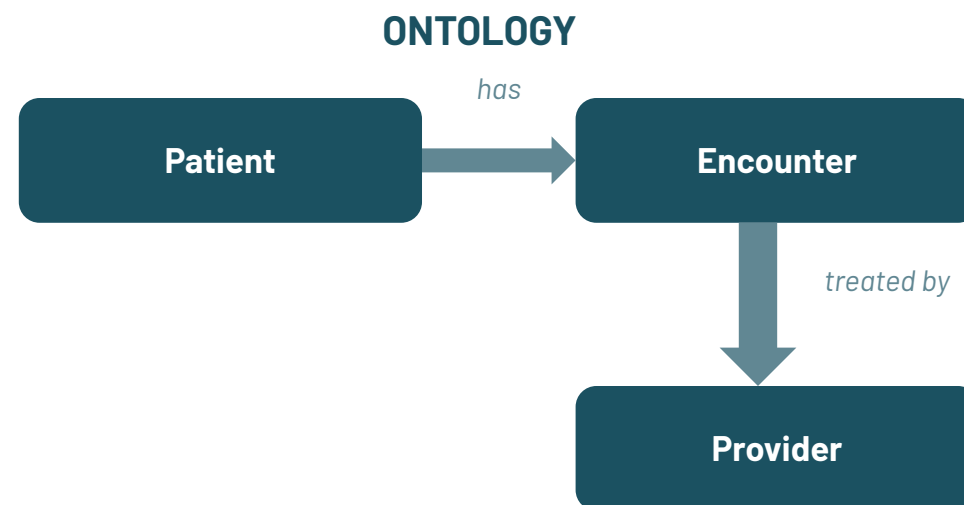
The business entities (Patient, Order...) and the typed relationships between them.

What “ontology” really means

A taxonomy is a tree; an ontology is a graph of typed relationships



one relation: is-a-kind-of



many typed relations

Real questions need rich relations – “which providers treated patients with diagnosis X?” A tree can’t express that; a graph can. That’s why dbxmetagen uses an ontology, not just categories.

Ontology: the same word, two depths

Formal ontology

A formal, explicit specification of a shared conceptualization (Gruber):

- Concepts (classes)
- Properties
- Relationships
- Rules / axioms a machine can reason over

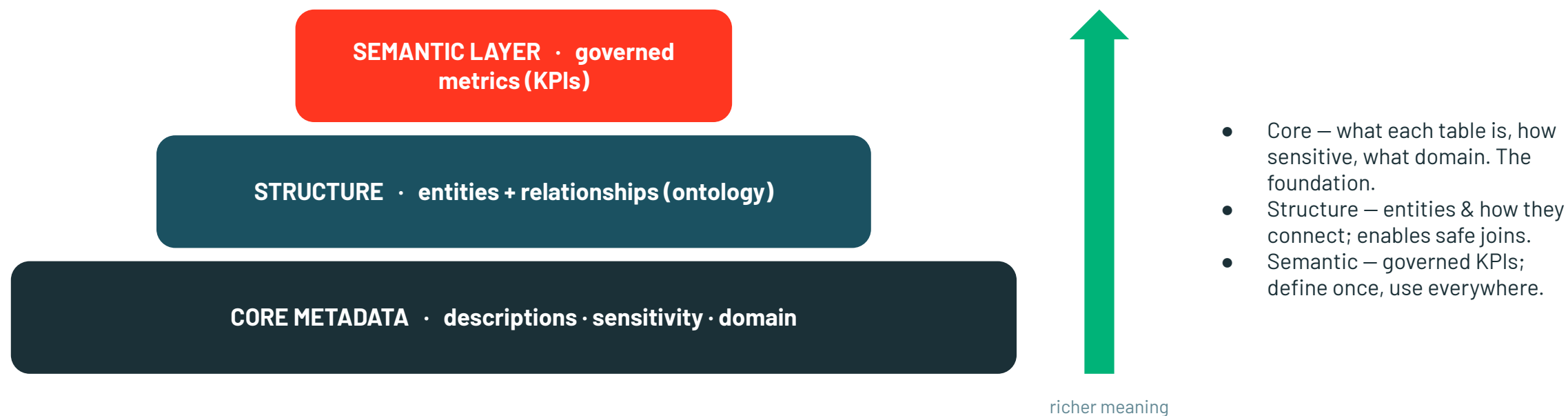
What dbxmetagen builds

A discovered, pragmatic model:

- Entities (Patient, Order...)
- The columns that describe them
- Relationships between entities
- Aligned to industry standards — but no formal reasoning

dbxmetagen does the discovery; formal axioms & reasoning are a different kind of tool.

One stack, three layers of richness



Each layer is built on the one below.

Three forms of the same knowledge

Same knowledge, increasingly meaningful: facts → connections → meaning



Organized facts about every table & column
— the tidy pile of facts.

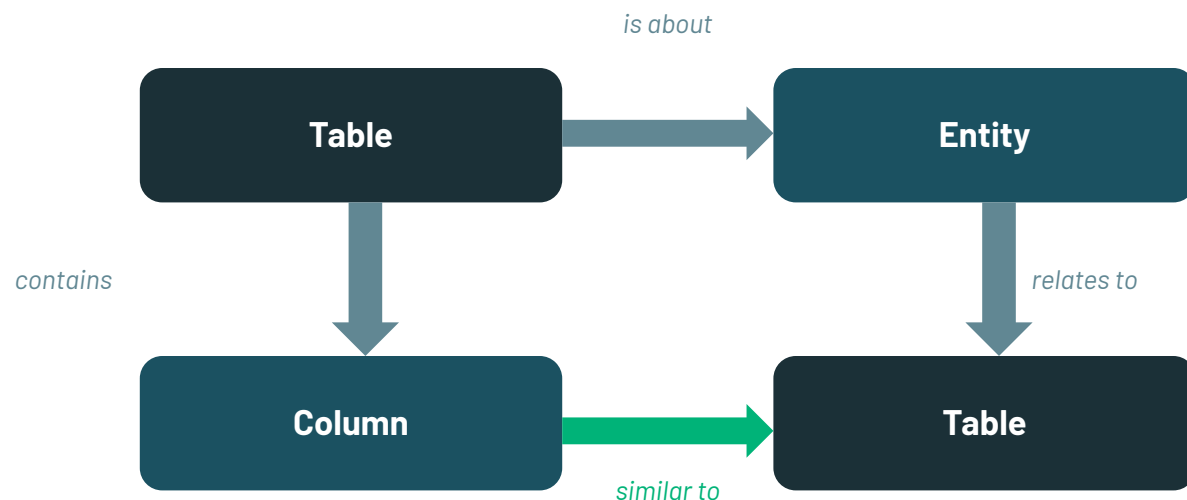
Those facts as a web you can traverse —
and find similar things.

The business entities & relationships
within it — the meaning.

The knowledge base is built first, as the foundation.
The graph and the ontology are built FROM it — not the other way around.

The knowledge graph, up close

Facts become a web you can walk – nodes joined by typed relationships



- Nodes = the things in your catalog: tables, columns, business entities.
- Edges = typed links: contains, is about, relates to, is similar to.
- You can TRAVERSE it – hop column → table → related tables to answer multi-hop questions.
- Similar things cluster into communities, making a huge catalog navigable.

The ontology is the business-meaning slice of this graph (entities + relationships).
The graph also captures structure (contains), similarity, and lineage – a richer web than the ontology alone.

Profiling: knowing the actual data

Statistics about the values – not the meaning, the shape

What it measures

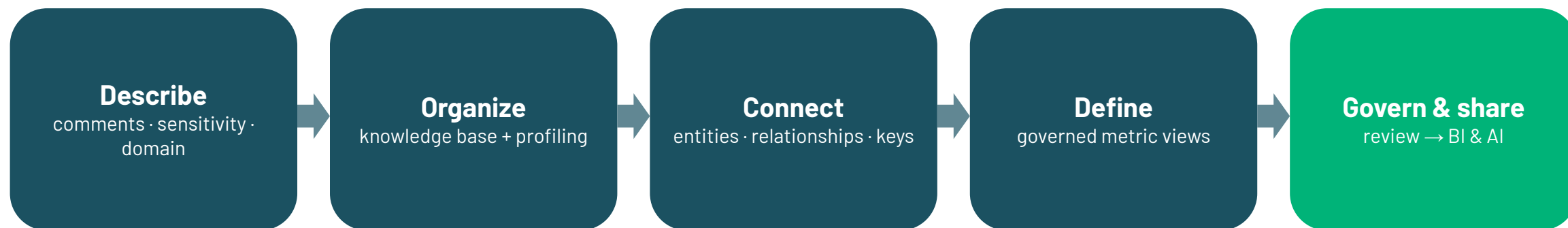
- How many nulls / % missing
- How many distinct values (cardinality)
- Min, max, ranges
- Value distributions
- → a data-quality score

Why it matters

- Gives the AI better context to write accurate descriptions
- Helps match foreign keys (compare value ranges & cardinality)
- Surfaces data-quality problems

Profiling looks at the SHAPE of the data; comments and the ontology capture its MEANING.
Together they make the documentation accurate.

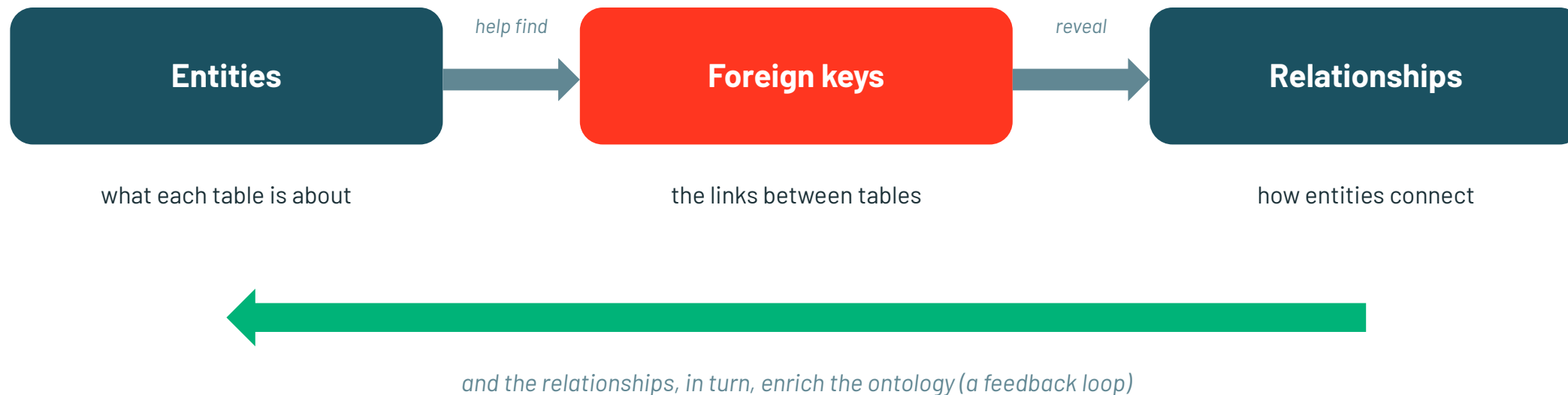
How dbxmetagen works, end to end



- Describe — AI writes plain-language comments, sensitivity labels, and business domains.
- Organize — consolidate everything into one clean knowledge base.
- Connect — build the graph, discover entities, infer the keys that join tables.
- Define — turn business questions into governed metrics; Govern — a human approves, then BI & AI consume.

Entities & relationships: a two-way street

You can name what a table is on its own; connecting tables needs their keys



Naming what a table IS needs only that table — its columns and description.
CONNECTING two tables needs a key. dbxmetagen predicts those keys, then a human confirms them — and the confirmed links feed back to make the ontology richer.

How a foreign key is inferred

No single clue is trusted alone – several signals, then a human



Names can hide a match (acct_holder vs customer). Values can mislead (two unrelated 6-digit IDs). Meaning catches what names and values miss. So dbxmetagen blends them – and a human has the final say.

What we mean by “meaning”

Three different kinds of evidence – meaning is the deepest

Name

The literal column name (cust_id).
Surface only – misses synonyms.

Data values

The actual contents. Can coincide by
accident, or mismatch across systems.

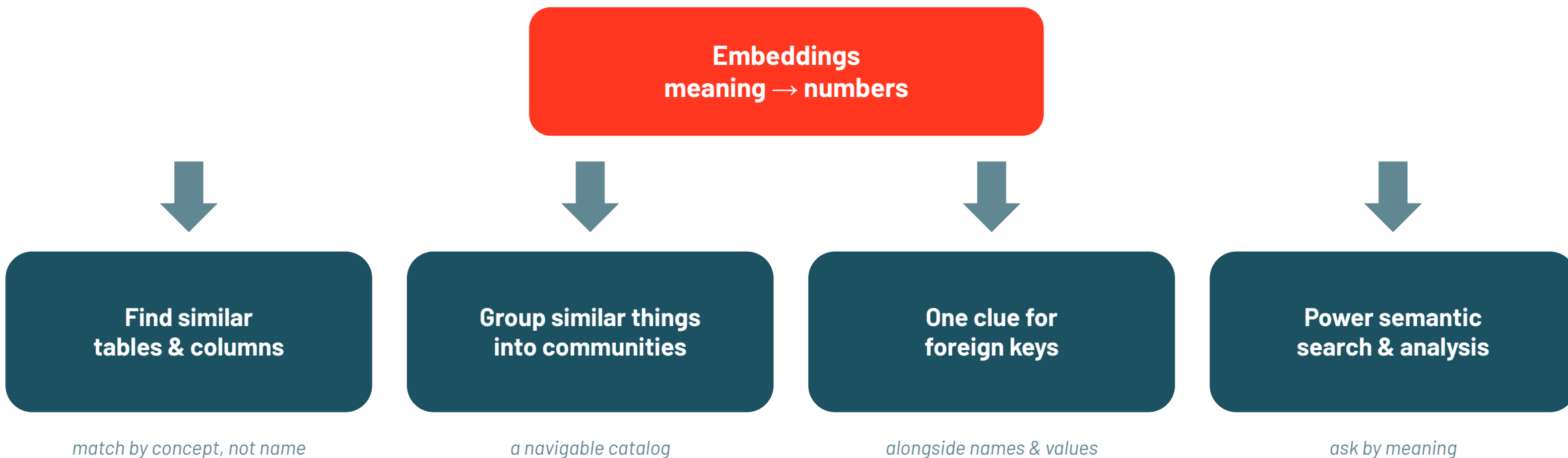
Meaning

The CONCEPT it represents, from its
description. Knows acct_holder =
customer = member.

Embeddings = “meaning turned into numbers.” Two columns whose concepts are close get nearby numbers, so synonyms are recognized even when names and values differ – and similar things get grouped together.
Example: “patient medical record number” and “identifier for the enrolled patient” mean nearly the same thing.

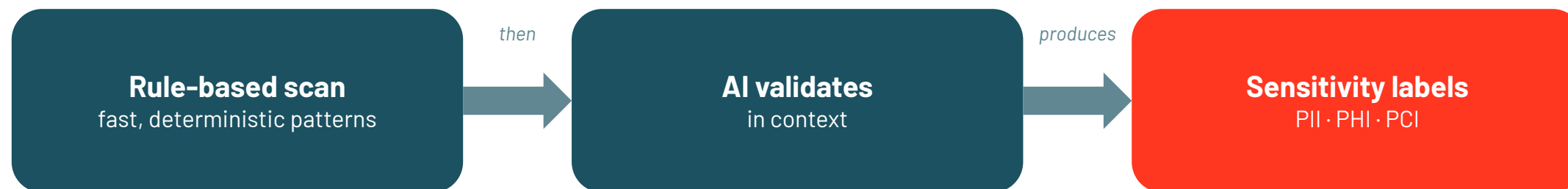
Where embeddings come into play

One idea – meaning turned into numbers – quietly powers four things



Embeddings add the "meaning" dimension – recognizing concepts that names and values miss.
They don't replace those signals; they complete them.

Finding sensitive data: rules + AI



Why two engines? Cheap, deterministic rules do the first pass over every column – near-zero cost. The AI then confirms in context (is this 9-digit number a social-security number, or an order id?). Result: accuracy across the whole catalog without paying to inspect every column by hand.

The semantic layer = meaning + metrics

Entities & relationships (ontology)

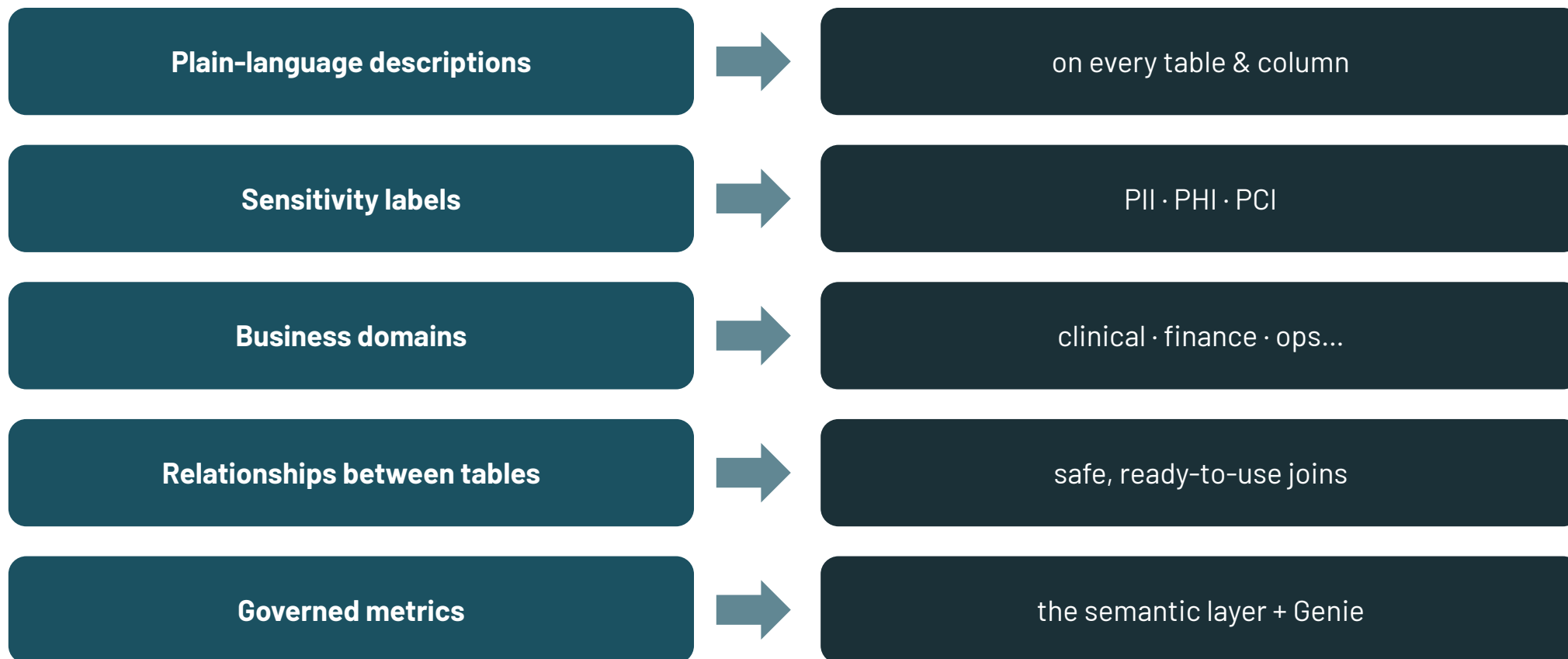
The business objects — Customer, Order, Patient — and how they connect. The “nouns” of your business.

Governed metrics (metric views)

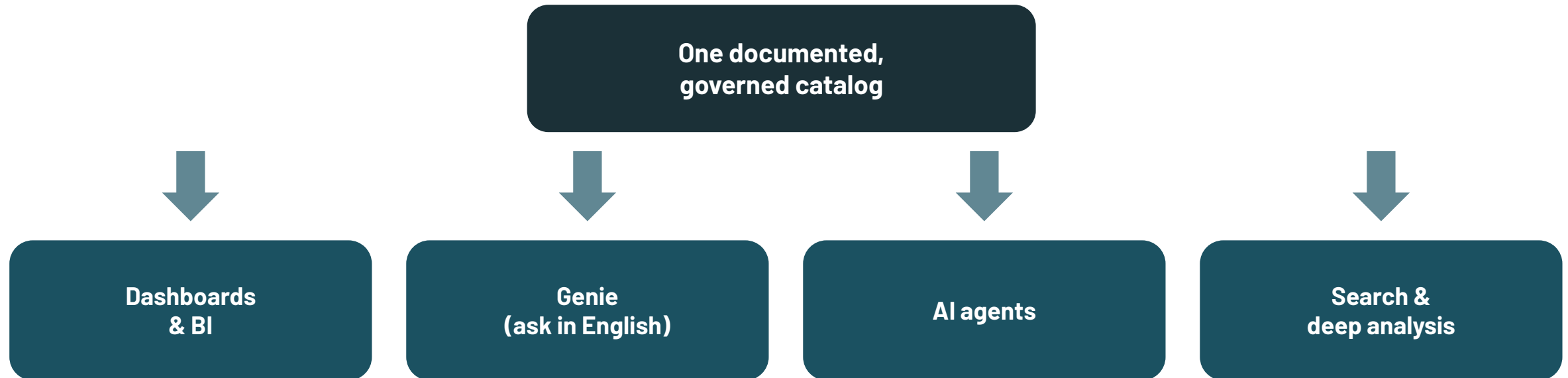
KPIs defined ONCE in the catalog — revenue, churn, length-of-stay — with their formula, dimensions, and joins.

A metric view defines a KPI once — so define “revenue” a single time, and every dashboard, query, and AI agent computes it identically. No more three dashboards showing three different revenue numbers. Together, the ontology (meaning) and the metrics sit on top of your documented catalog — that is the semantic layer.

What you get – all inside your catalog



One source of truth, every consumer



- Dashboards & BI – built on documented tables and shared metric definitions.
- Genie – business users ask questions in plain English and get trustworthy answers.
- AI agents – act without being told which table or column to use.
- Search & deep analysis – traverse the knowledge graph to answer connected questions.

dbxmetagen vs a formal ontology tool

Complementary: discover the draft, then formalize it

dbxmetagen (discovery)

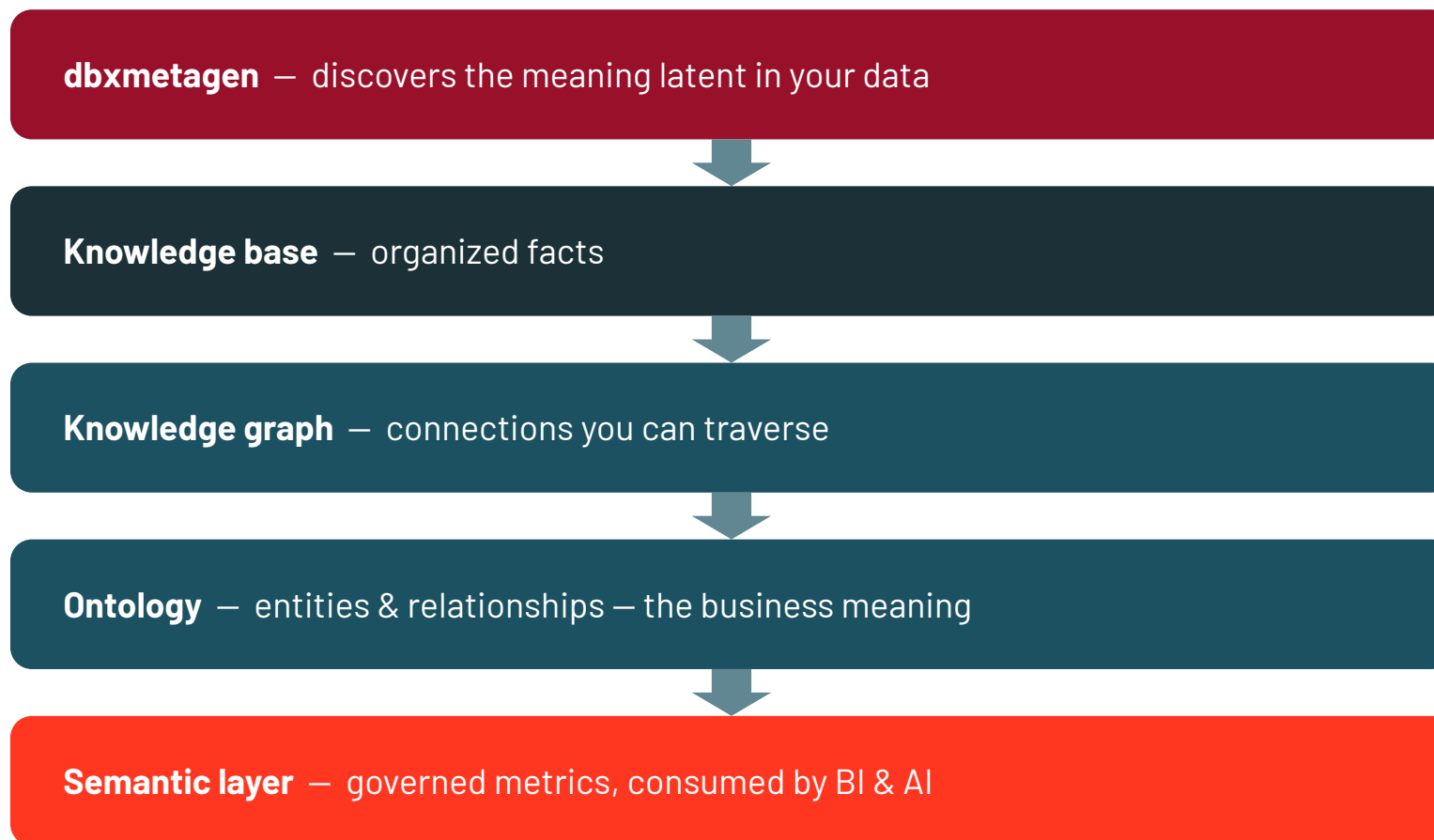
- Discovers meaning with AI
- Works at the concept / type level
- Describes what it finds
- Fast, approximate draft
- Always human-reviewed

Formal ontology (authoring)

- You design the model by hand
- Works down to individual records
- Reasons & enforces rules
- Rigorous and exact
- Governed by design

Use dbxmetagen to bootstrap the draft; formalize and govern it in a dedicated ontology tool.

How it all fits together

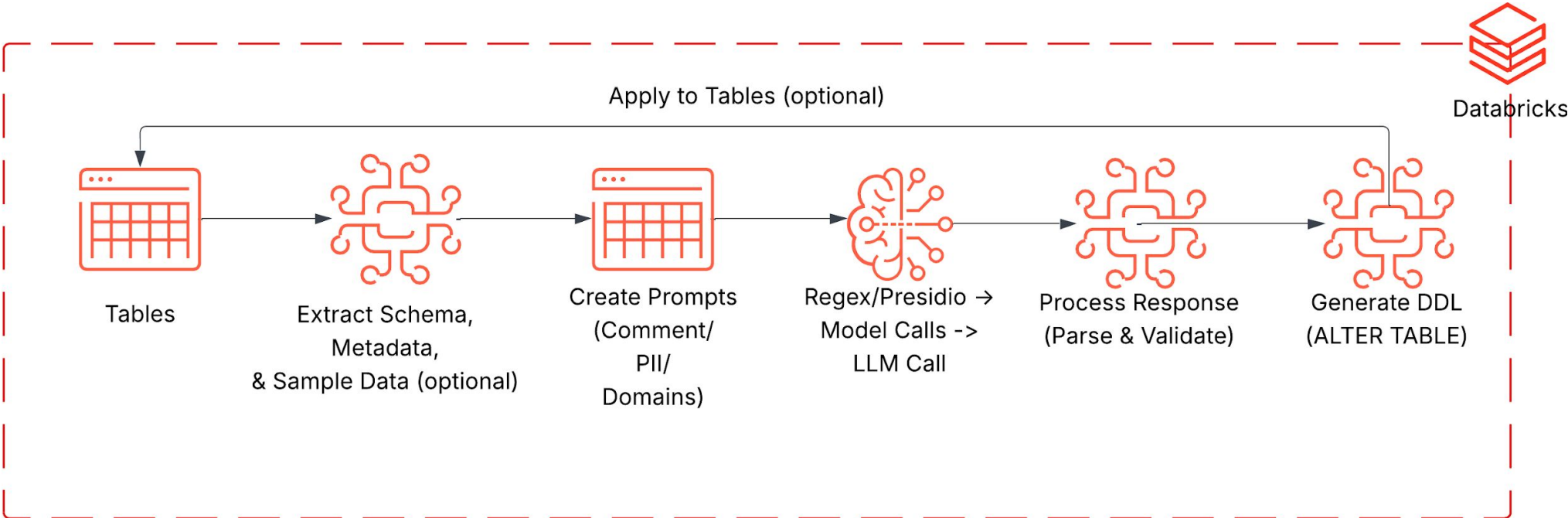


How it works on Databricks

Architecture · jobs · the app · permissions · MCP · deployment

Inside the pipeline: from table to applied metadata

Two phases, end to end – generate metadata, then analyze it – all within Databricks



Phase 1 – generate comments, PII, and domain per table (deterministic rules + LLM → parse & validate → optional ALTER TABLE). Phase 2 – aggregate into knowledge bases, then build graph, embeddings, ontology, profiling, and FK predictions.

What it builds: five metadata layers

Each layer is a set of Delta tables, built on the one below it

Layer	Key tables	Purpose
Knowledge Base	table / column / schema_knowledge_base, extended_metadata	Aggregated metadata from LLM outputs + system tables – the source of truth
Profiling	profiling_snapshots, column_profiling_stats, data_quality_scores	Statistical profiling and data-quality scoring
Graph	graph_nodes, graph_edges, clusters, community_summaries	Similarity edges, K-means clustering, AI community summaries
Ontology	ontology_entities, column_properties, relationships, chunks	Business entities, column roles, typed relationships
Vector Index	metadata_vs_index, ontology_vs_index	Hybrid semantic search over metadata and entities

All downstream consumers read from the knowledge base – not Unity Catalog. Applying DDL to UC is an optional publishing step.

Orchestrated as Databricks Asset Bundle jobs

Deployed with `deploy.sh` — jobs run as the deployer's identity

Phase 1 · Generate

`metadata_generator_job` — single mode
`metadata_parallel_modes_job` — all 3 modes

Modes: comment · PII · domain
→ written to `metadata_generation_log`
`apply_ddl` optional (write back to UC)

Phase 2 · Analytics

`full_analytics_pipeline_job` —
KB · graph · embeddings · profiling ·
ontology · similarity · FK prediction

Plus: `semantic_layer_job`,
`build_vector_index_job`, `sync_graph_lakebase_job`

Run the full app experience, jobs-only (`--no-app`), or pip-install the library on any cluster for core generation.

The dashboard app: Design · Review · Explore

FastAPI + React, deployed via DAB — the full metadata lifecycle in one place

Design

Generate Metadata — core + advanced jobs, model selection, Customer Context

Define Metrics — metric views, KPI Library

Build Genie Space — instructions + example SQL

Review

Review & Apply — browse, edit, approve, and apply metadata back to Unity Catalog

Coverage — schema-wide completeness and review-status metrics

Explore

Agent — GraphRAG deep analysis, graph explorer, semantic search, MLflow traces

Entity Browser — entity-first navigation with conformance view

The app's service principal needs only SELECT to power the UI — generation and apply run as separate, governed jobs.

Two identities, by design

Reading the catalog and writing to it are deliberately separated

App service principal – READS

Controls what the app UI can read.

Needs SELECT on your catalog to browse tables, coverage, metadata, and graph data.

UC grants applied automatically on every deploy.

Deployer identity – WRITES

Controls what jobs can write.

Jobs run as the user who ran deploy.sh.

Needs CREATE TABLE, ALTER TABLE, and SET TAGS to generate and apply metadata.

A user can view metadata in the app even without permission to generate or apply it – and vice versa. OBO mode also supported.

Four ways to run it

From bulk auto-apply to human-in-the-loop governance

1 · Generate & Apply

`apply_ddl=true`. Comments written and immediately applied to UC.

Best for: initial bulk generation.

2 · Generate Only

`apply_ddl=false`. Populates the internal KB; UC untouched.

Best for: downstream features without UC writes.

3 · Review, then Apply

Generate → human reviews in the UI or TSV → apply approved.

Best for: enterprise sign-off.

4 · Edit in UI

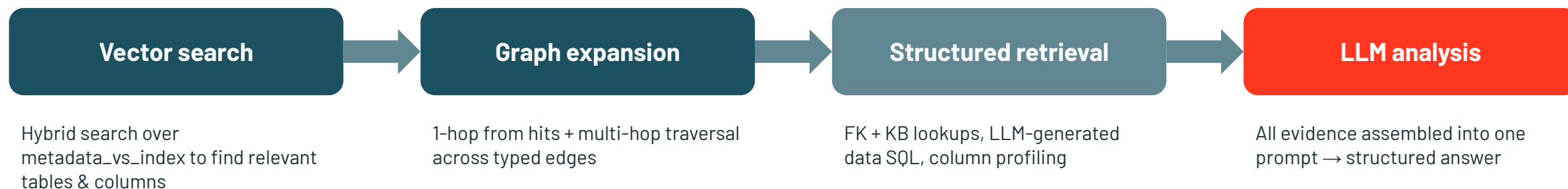
Edit comments / domains in the UI → saved to the KB.

Best for: iterative quality refinement.

Human edits are protected automatically — a timestamp comparison means routine KB rebuilds never overwrite your changes.

Ask in plain English: the GraphRAG agent

A deterministic evidence-gathering pipeline runs before a single LLM call



The same data assets are exposed through MCP — any MCP client can run the same queries. Results include MLflow trace links.

Query your metadata from any AI client (MCP)

Knowledge base, graph, and vector index exposed as Databricks Managed MCP servers

Vector Search MCP

Hybrid (semantic + keyword) search over metadata_vs_index.

Discovers relevant tables, columns, entities, FKs, and metric views.

UC Functions MCP

9 governed SQL functions over the KB and knowledge graph:

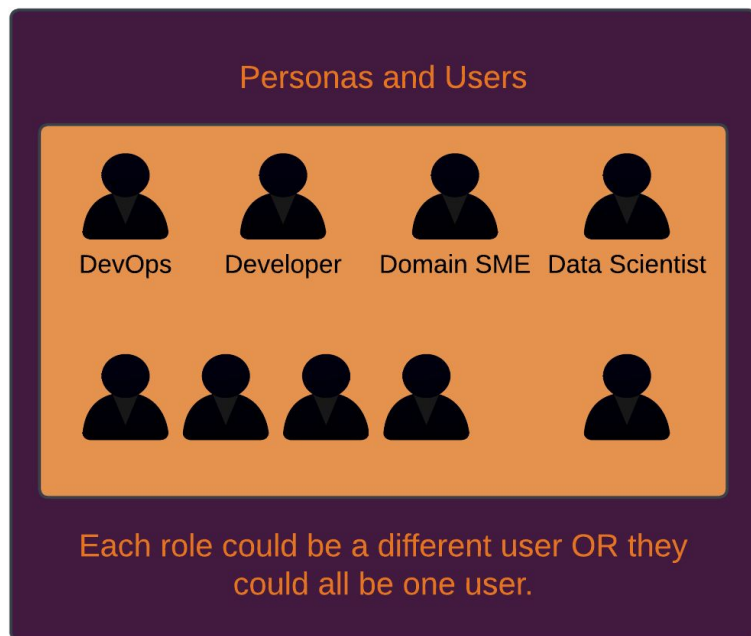
get_table_metadata · search_knowledge_base · get_fk_predictions ·
query_graph_nodes · traverse_graph (multi-hop)...

Clients: Cursor · Claude Code · AI Playground · custom agents | Governed by Unity Catalog + on-behalf-of-user auth

MCP support is under active development — the same data assets the dashboard's deep-analysis agent uses today.

Who uses it & getting started

One tool, many roles – and three ways to bring it into a project



1 · Clone + `deploy.sh`

Full deployment – all jobs + the web app (Asset Bundle).

2 · UC Volume wheel + recipes

Production: publish the wheel + portable recipe YAMLs to shared locations.

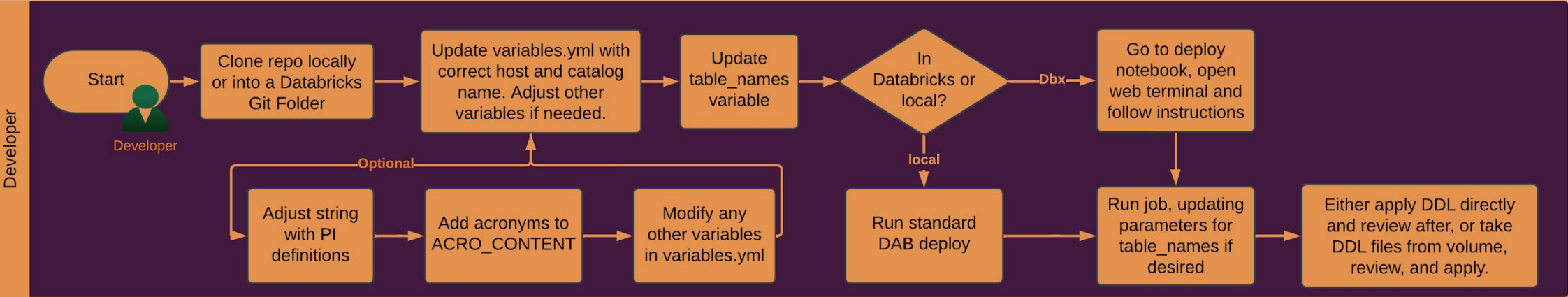
3 · pip install from GitHub

Prerequisites: Unity Catalog, a SQL warehouse, and a Foundation Model endpoint (e.g. databricks-claude-sonnet-4-6).

Deploying it: the simplest path

One developer, via Databricks Asset Bundles – clone, configure, deploy, run, review

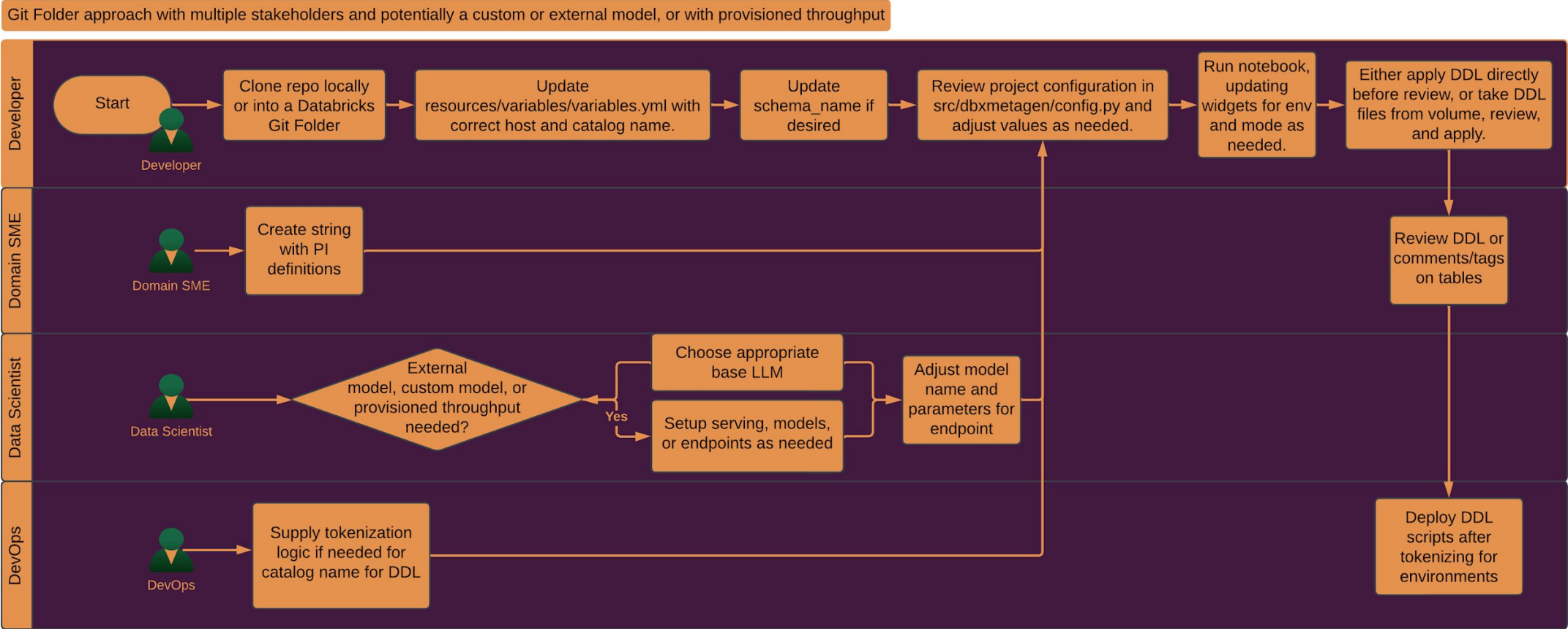
Simplest workflow - Databricks Workflow only - Asset bundle deploy



Set host + catalog in variables.yml → deploy the bundle → run the job → apply DDL or export for review. Optional: add PII definitions and acronyms (ACRO_CONTENT).

Deploying it: multiple stakeholders

A Git Folder workflow spanning Developer · Domain SME · Data Scientist · DevOps



Why it matters

FTE-years
→ **hours**

Documentation at enterprise scale

BI + AI

from one governed definition

Bootstrap

metrics, ontology & Genie, fast



databricks

Conceptual + implementation overview · Open source: github.com/databricks-industry-solutions/dbxmetagen